

Name: Bestha Akshaya

Course code: CSA0611

Regno:192424336

Course name: DAA

Slot C

LAB EXPERIMENTS

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Code:

```
words=["abc","car","ada","racecar","cool"]
```

```
for i in words:
```

```
    if i==i[::-1]:
```

```
        print(i)
```

OUTPUT:

The screenshot shows a code editor with a file named 'main.py'. The code contains a list 'words' with elements 'abc', 'car', 'ada', 'racecar', and 'cool'. A 'for' loop iterates over each word in 'words'. Inside the loop, an 'if' statement checks if the word is equal to its reverse (i == i[::-1]). If true, the word is printed. The output window on the right shows 'ada' and 'racecar' printed on separate lines, followed by the message '=== Code Execution Successful ==='. The editor interface includes icons for file operations, a settings gear, a 'Share' button, and a blue 'Run' button.

```
1 words=["abc","car","ada","racecar","cool"]
2 for i in words:
3     if i==i[::-1]:
4         print(i)
```

ada
racecar
=== Code Execution Successful ===

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively.

Calculate the following values: answer1 : the number of indices i such that nums1[i]

exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1

Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Code:

```
nums1 = [2, 3, 2]
nums2 = [1, 2]
answer1 = 0
answer2 = 0
for i in nums1:
    if i in nums2:
        answer1 += 1
for i in nums2:
    if i in nums1:
        answer2 += 1
print([answer1, answer2])
```

OUTPUT:

```
main.py
1 nums1 = [2, 3, 2]
2 nums2 = [1, 2]
3 answer1 = 0
4 answer2 = 0
5 for i in nums1:
6     if i in nums2:
7         answer1 += 1
8 for i in nums2:
9     if i in nums1:
10        answer2 += 1
11 print([answer1, answer2])
```

Output

```
[2, 1]
=== Code Execution Successful ===
```

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Code:

```
nums = [1, 2, 1]
ans = 0
for i in range(len(nums)):
    s = set()
    for j in range(i, len(nums)):
        s.add(nums[j])
        ans += len(s) ** 2
print(ans)
```

OUTPUT:

```
main.py
1 nums = [1, 2, 1]
2 ans = 0
3 for i in range(len(nums)):
4     s = set()
5     for j in range(i, len(nums)):
6         s.add(nums[j])
7         ans += len(s) ** 2
8 print(ans)
```

Output

15

=== Code Execution Successful ===

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 = 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 = 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 = 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 = 12$, which is divisible by 2.

Code:

```
nums = [3, 1, 2, 2, 2, 1, 3]
k = 2
count = 0
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1
print(count)
```

OUTPUT:

```
main.py
1 nums = [3, 1, 2, 2, 2, 1, 3]
2 k = 2
3
4 count = 0
5
6 for i in range(len(nums)):
7     for j in range(i + 1, len(nums)):
8         if nums[i] == nums[j] and (i * j) % k == 0:
9             count += 1
10
11 print(count)
```

4

=== Code Execution Successful ===

5. Write a program FOR THE BELOW TEST CASES with least time complexity

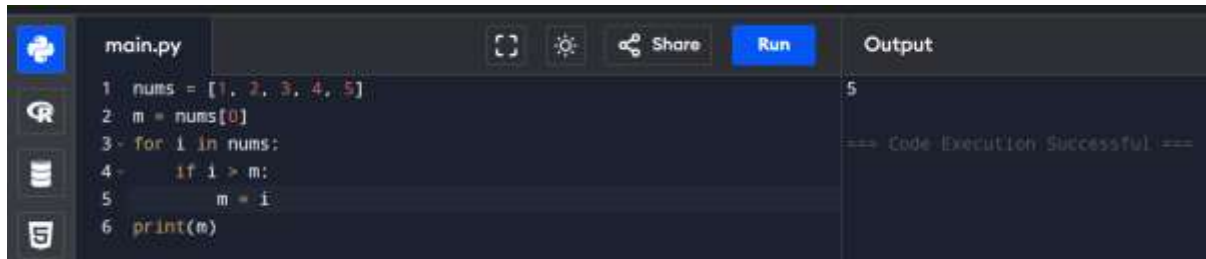
Test Cases: -

- 1) Input: {1, 2, 3, 4, 5}
- 2) Expected Output: 5

Code:

```
nums = [1, 2, 3, 4, 5]
m = nums[0]
for i in nums:
    if i > m:
        m = i
print(m)
```

OUTPUT:



```
main.py
1 nums = [1, 2, 3, 4, 5]
2 m = nums[0]
3 for i in nums:
4     if i > m:
5         m = i
6 print(m)
```

Output

5

=== Code Execution Successful ===

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List

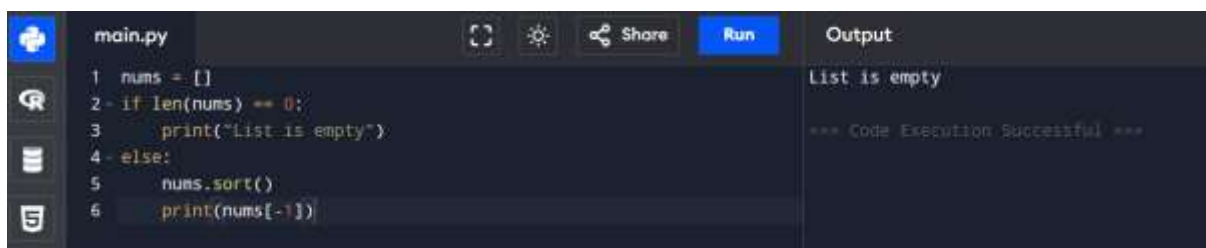
1. Input: []

2. Expected Output: None or an appropriate message indicating that the list is empty.

Code:

```
nums = []
if len(nums) == 0:
    print("List is empty")
else:
    nums.sort()
    print(nums[-1])
```

OUTPUT:



```
main.py
1 nums = []
2 if len(nums) == 0:
3     print("List is empty")
4 else:
5     nums.sort()
6     print(nums[-1])
```

Output

List is empty

=== Code Execution Successful ===

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

- Input: [3, 7, 3, 5, 2, 5, 9, 2]
- Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Code:

```
nums = [3, 7, 3, 5, 2, 5, 9, 2]
```

```
unique = []
```

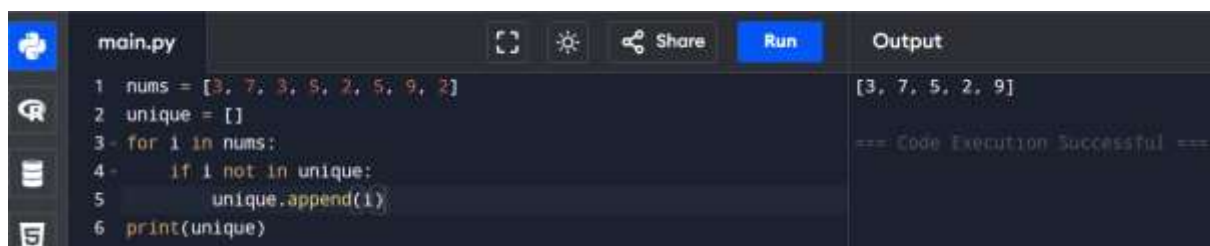
```
for i in nums:
```

```
    if i not in unique:
```

```
        unique.append(i)
```

```
print(unique)
```

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 nums = [3, 7, 3, 5, 2, 5, 9, 2]
2 unique = []
3 for i in nums:
4     if i not in unique:
5         unique.append(i)
6 print(unique)
```

The 'Output' pane on the right shows the result: [3, 7, 5, 2, 9]. Below the output, it says '=== Code Execution Successful ==='.

8. Sort an array of integers using the bubble sort technique. Analyse its time complexity using Big-O notation. Write the code

Code:

```
nums = [5, 3, 8, 4, 2]
```

```
for i in range(len(nums)):
```

```
    for j in range(len(nums)-1-i):
```

```
        if nums[j] > nums[j+1]:
```

```
            nums[j], nums[j+1] = nums[j+1], nums[j]
```

```
print(nums)
```

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 nums = [5, 3, 8, 4, 2]
2 for i in range(len(nums)):
3     for j in range(len(nums)-1-i):
4         if nums[j] > nums[j+1]:
5             nums[j], nums[j+1] = nums[j+1], nums[j]
6 print(nums)
```

The 'Output' pane on the right shows the result: [2, 3, 4, 5, 8]. Below the output, it says '=== Code Execution Successful ==='.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Code:

```
arr = [3, 4, 6, -9, 10, 8, 9, 30]
```

```
key = 10
```

```
arr.sort()
```

```
low = 0
```

```
high = len(arr) - 1
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        print("Element", key, "is found at position", mid + 1)
```

```
        break
```

```
    elif arr[mid] < key:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
else:
```

```
    print("Element not found")
```

OUTPUT:



```
main.py
1: arr = [3, 4, 6, -9, 10, 8, 9, 30]
2: key = 10
3: arr.sort()
4: low = 0
5: high = len(arr) - 1
6: while low <= high:
7:     mid = (low + high) // 2
8:     if arr[mid] == key:
9:         print("Element", key, "is found at position", mid + 1)
10:        break
11:    elif arr[mid] < key:
12:        low = mid + 1
13:    else:
14:        high = mid - 1
15: else:
16:     print("Element not found")
```

Output: Element 10 is found at position 7

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

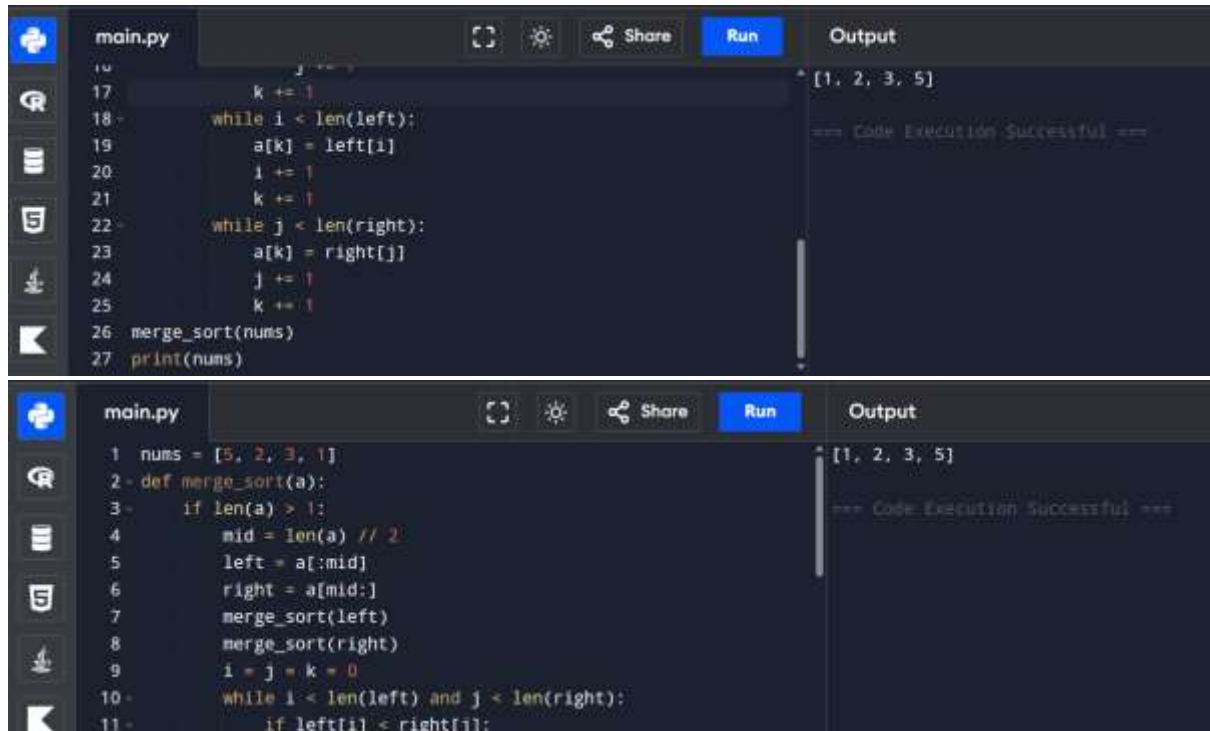
Code:

```
nums = [5, 2, 3, 1]

def merge_sort(a):
    if len(a) > 1:
        mid = len(a) // 2
        left = a[:mid]
        right = a[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            a[k] = right[j]
            j += 1
            k += 1
    merge_sort(nums)
```


print(nums)

OUTPUT:



The image displays two screenshots of a code editor interface, likely VS Code, showing the implementation and execution of a merge sort algorithm. The editor has a dark theme and includes a sidebar with icons for Explorer, Search, and Run and Debug. The top screenshot shows the final steps of the merge sort process, where the sorted array [1, 2, 3, 5] is printed. The bottom screenshot shows the initial setup of the merge sort function, including the definition of the merge_sort function and the initial array [5, 2, 3, 1].

Top Screenshot:

```
main.py
16         j += 1
17         k += 1
18     while i < len(left):
19         a[k] = left[i]
20         i += 1
21         k += 1
22     while j < len(right):
23         a[k] = right[j]
24         j += 1
25         k += 1
26 merge_sort(nums)
27 print(nums)
```

Output:

```
[1, 2, 3, 5]
=== Code Execution Successful ===
```

Bottom Screenshot:

```
main.py
1 nums = [5, 2, 3, 1]
2 def merge_sort(a):
3     if len(a) > 1:
4         mid = len(a) // 2
5         left = a[:mid]
6         right = a[mid:]
7         merge_sort(left)
8         merge_sort(right)
9         i = j = k = 0
10        while i < len(left) and j < len(right):
11            if left[i] < right[j]:
```

Output:

```
[1, 2, 3, 5]
=== Code Execution Successful ===
```